

The Ultimate Web Development Handbook



Introduction to Node.js

Why Do We Need a Backend?

In web development, **frontend** and **backend** are two key parts of an application.

- The **frontend** is what users see and interact with—like buttons, forms, and text on a webpage. It runs in the browser.
- The **backend** is like the behind-the-scenes part. It handles things like:
 - Storing and retrieving data from databases
 - Authenticating users (login/signup)
 - Processing business logic
 - Securing sensitive operations
 - Handling API requests from the frontend

Without a backend, a website can't store user data, communicate with a database, or perform secure tasks. The frontend would just be a static page with limited interactivity.

Example

Imagine a to-do list app:

- The **frontend** displays the tasks and lets the user add or remove them.
 - The **backend** saves these tasks to a database, so they're still there when the user comes back later.
-

Introduction to Node.js

Node.js is a runtime environment that lets you run JavaScript on the **server**, not just in the browser.

- Normally, JavaScript runs only in the browser (client-side).
- Node.js allows you to run JavaScript on your computer or server (server-side).

With Node.js, you can build the backend of your application using JavaScript—the same language you use for the frontend. This makes development faster and easier, especially for beginners.

Key Features of Node.js

- Built on Chrome's V8 JavaScript engine
 - Handles many requests at once using non-blocking (asynchronous) code
 - Has a large ecosystem of libraries (called npm)
 - Great for building APIs, real-time apps (like chat apps), and full-stack JavaScript projects
-

Server-side vs. Client-side JavaScript

Feature	Client-side (Browser)	Server-side (Node.js)
Runs on	User's browser	Web server
Use case	Displaying content, UI interaction	Storing data, handling logic
Access to system resources	Limited (sandboxed)	Full access (file system, network)
Performance	Depends on user's device	Depends on server
Examples	Form validation, animations	Database queries, user authentication

Why the Difference Matters

- Running JavaScript in the **client** is good for creating interactive webpages, but it's limited and not secure for sensitive operations.
 - Running JavaScript in the **server** (with Node.js) allows you to perform secure tasks and manage application logic centrally.
-

Summary

- A **backend** is essential for dynamic websites that need to store data, handle users, or connect to databases.
- **Node.js** lets you write backend code using JavaScript, making full-stack development more accessible.
- **Client-side** JavaScript is for user interaction; **server-side** JavaScript (via Node.js) handles the logic and data behind the scenes.

Installing Node.js and npm

To run JavaScript on the server and build backend applications with Node.js, you first need to install **Node.js**. When you install Node.js, **npm** (Node Package Manager) is installed automatically.

Step 1: Download Node.js

1. Go to the official Node.js website: <https://nodejs.org>
2. You'll see two versions:
 1. **LTS (Long-Term Support)**: Recommended for most users. It's stable and reliable.
 2. **Current**: Has the latest features but may not be as stable.
3. Download the **LTS** version for your operating system (Windows, macOS, or Linux).

Step 2: Install Node.js On Windows and macOS

1. Run the installer you downloaded.
2. Follow the setup instructions:
 1. Accept the license agreement.
 2. Choose the default options.
3. After installation, Node.js and npm will be available globally on your system.

Step 3: Verify Installation

After installation, open a terminal (Command Prompt, Terminal, or shell) and check the versions:

```
node -v
```

This prints the installed Node.js version.

```
npm -v
```

This prints the installed npm version.

If you see version numbers for both, the installation was successful.

What is npm?

npm (Node Package Manager) is a tool that comes with Node.js. It allows you to:

- Install open-source packages and libraries (called “modules”)
- Manage project dependencies
- Run scripts for building, testing, and starting your app

You’ll use npm often when building projects with Node.js.

Summary

- Install Node.js from <https://nodejs.org>
- Node.js includes npm, which is used to manage packages
- Use your terminal to verify the installation with `node -v` and `npm -v`

You’re now ready to start building backend applications with JavaScript and Node.js. Let me know if you want a guide on starting your first Node.js project.

Using npm Packages in Node.js (with Express)

In this guide, we'll install and use an npm package in a Node.js project. We'll use **Express**, a popular web framework for Node.js. Don't worry about the details of Express for now—we'll cover that later. The goal here is simply to show how to install and use packages with npm.

Step 1: Initialize a New Project

Create a new project folder and initialize it:

```
mkdir my-npm-app  
cd my-npm-app  
npm init -y
```

This creates a `package.json` file that keeps track of your project's dependencies.

Step 2: Install Express

Use `npm install` to install the Express package:

```
npm install express
```

After installing, you'll see a `node_modules` folder and a `package-lock.json` file created. The `package.json` file will also list Express under `dependencies`.

Step 3: Create a Simple Server with Express

Create a new file called `app.js` :

```
touch app.js
```

Add the following code to `app.js` :

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

const port = 3000;
app.listen(port, () => {
  console.log(`Server is running at http://localhost:${port}`);
});
```

We'll explore what this code does later. For now, it just starts a basic server.

Step 4: Run Your App

To start the app, run:

```
node app.js
```

Visit <http://localhost:3000> in your browser. You should see:

```
Hello from Express!
```

Step 5: Use Node.js Watch Mode

Node.js now supports **watch mode** natively (from version 18 and above)

To run your app in watch mode:

```
node --watch app.js
```

This means Node.js will automatically restart whenever you save changes to `app.js` or other imported files.

Note: Watch mode works best in modern versions of Node.js. You can check your Node.js version using:

```
node -v
```

Step 6: Uninstall a Package

If you ever want to remove a package (like Express), use:

```
npm uninstall express
```

This removes it from `node_modules` and updates your `package.json`.

Summary

- Initialized a Node.js project using `npm init -y`
- Installed an npm package (`express`)
- Used it in a basic server file (`app.js`)
- Ran the app using both normal and watch mode (`node` and `node --watch`)
- Learned how to uninstall packages with `npm uninstall`

Creating a Simple Node.js Application

Now that Node.js and npm are installed, you can create your first Node.js application. This guide walks you through building a basic "Hello, World" server.

Step 1: Create a New Project Folder

Open your terminal and create a new folder for your project:

```
mkdir my-node-app  
cd my-node-app
```

Step 2: Initialize the Project

Run the following command to create a `package.json` file, which keeps track of your project settings and dependencies:

```
npm init -y
```

This generates a basic `package.json` file with default values.

Step 3: Create the Application File

Create a new file named `app.js` (or any name you prefer):

```
touch app.js
```

Open `app.js` in your code editor and add the following code:

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!\n');
});

const port = 3000;
server.listen(port, () => {
  console.log(`Server running at http://localhost:${port}/`);
});
```

This creates a basic HTTP server that responds with "Hello, World!" to every request.

Step 4: Run the Application

In your terminal, start the server:

```
node app.js
```

If everything is set up correctly, you should see this message in the terminal:

```
Server running at http://localhost:3000/
```

Open your web browser and visit <http://localhost:3000>. You should see "Hello, World!" displayed.

Summary

- You created a new folder and initialized a Node.js project.
- You wrote a simple server using Node's built-in `http` module.
- You started the server and accessed it in your browser.

Node.js Modules

Modules in Node.js are reusable pieces of code that help organize programs into separate files and components.

Types of Modules

1. Core Modules

Built into Node.js, no need to install.

Example: `fs` , `http` , `path`

```
const fs = require('fs');
const data = fs.readFileSync('file.txt', 'utf8');
console.log(data);
```

2. Local Modules Custom modules created in your project.

Example: `math.js`

```
// math.js
function add(a, b) {
  return a + b;
}

module.exports = { add };
```

```
// app.js
const math = require('./math');
console.log(math.add(5, 3));
```

3. Third-party Modules Installed via npm (Node Package Manager). Example:

`express` , `lodash`

```
npm install express
```

```
const express = require('express');  
const app = express();  
  
app.get('/', (req, res) => {  
  res.send('Hello World');  
});  
  
app.listen(3000);
```

Exporting and Importing

- Use `module.exports` to export
- Use `require()` to import

Summary

Node.js modules keep code organized, reusable, and maintainable. Whether you're using built-in modules, custom files, or third-party packages, modules are fundamental to every Node.js project.

ES6 Modules vs CommonJS in Node.js

Node.js supports two main module systems:

1. **CommonJS (CJS)** – Default in older Node.js versions
 2. **ES6 Modules (ESM)** – Modern standard, used in frontend and supported natively in recent Node.js versions
-

1. CommonJS

- File Extension: `.js`
- Import Syntax: `require()`
- Export Syntax: `module.exports` or `exports`

Example:

```
// math.js
function add(a, b) {
  return a + b;
}
module.exports = { add };
```

```
// app.js
const math = require('./math');
console.log(math.add(2, 3));
```

2. ES6 Modules

- File Extension: `.mjs` or `.js` with `"type": "module"` in `package.json`

- Import Syntax: `import`
- Export Syntax: `export` / `export default`

Example:

```
// math.mjs
export function add(a, b) {
  return a + b;
}
```

```
// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3));
```

Key Differences

Feature	CommonJS (CJS)	ES6 Modules (ESM)
Syntax	<code>require</code> , <code>module.exports</code>	<code>import</code> , <code>export</code>
File extension	<code>.js</code>	<code>.mjs</code> or <code>.js</code> with <code>"type": "module"</code>
Loading style	Synchronous	Asynchronous
Support	Default in Node.js	Modern Node.js (14+)
Top-level await	Not supported	Supported
Used in	Node.js traditionally	Both frontend and backend

When to Use What

- Use **CommonJS** if you're working on older Node.js projects or need compatibility with legacy packages.
 - Use **ES6 Modules** for new projects to align with modern JavaScript standards and browser support.
-

Mixing CJS and ESM

Mixing module types can be tricky:

- You can't use `require()` to import an ES6 module.
 - You can't use `import` to load a CommonJS module unless it's default-exported.
-

Conclusion

Both module systems help organize and reuse code, but **ES6 modules are the future**, offering cleaner syntax and better interoperability across frontend and backend.

Understanding the Node.js Wrapper Function and Special Variables

In Node.js, every JavaScript file is wrapped inside a special function before it is executed. This allows each file to have its own private scope, preventing variables from leaking into the global scope.

The Wrapper Function

Node.js wraps your code like this internally:

```
(function(exports, require, module, __filename, __dirname) {  
  // Your entire file code lives here  
});
```

This is known as the **Module Wrapper Function**. Because of this, your Node.js file gets access to the following special variables:

Special Variables in Node.js

`__filename`

- Returns the absolute path of the current file.

```
console.log(__filename);  
// Example: /Users/haris/project/app.js
```

`__dirname`

- Returns the absolute path of the directory that contains the current file.

```
console.log(__dirname);  
// Example: /Users/haris/project
```

exports and module.exports

- Used to export variables or functions from a module.

require

- Used to import modules.

module

- Represents the current module and contains metadata about it.
-

Why This Matters

This wrapper allows each file to be treated as a separate module. It ensures:

- Code isolation (no global scope pollution)
 - Access to helpful variables like `__dirname` and `__filename`
 - A consistent module system using CommonJS
-

Example

```
// test.js  
console.log('Filename:', __filename);  
console.log('Directory:', __dirname);
```

When you run this with `node test.js`, it will print the full path of the file and its directory.

Understanding the wrapper function and special variables is key to mastering how Node.js modules work internally.